

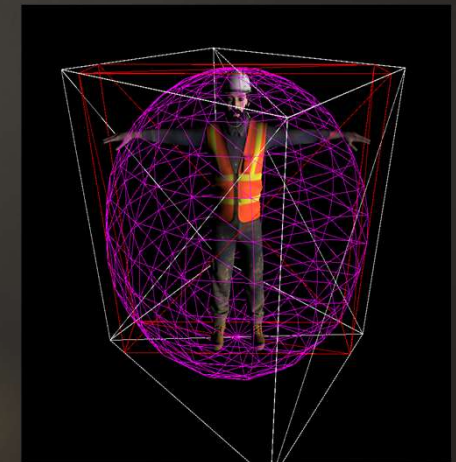
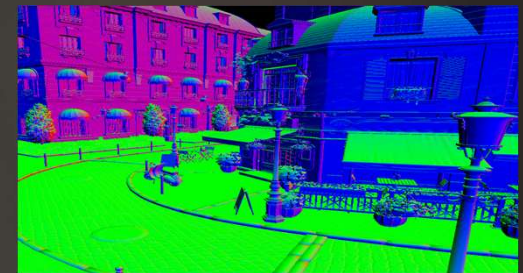
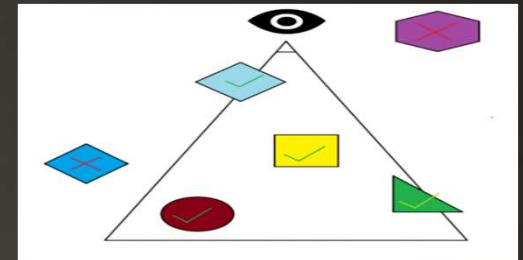
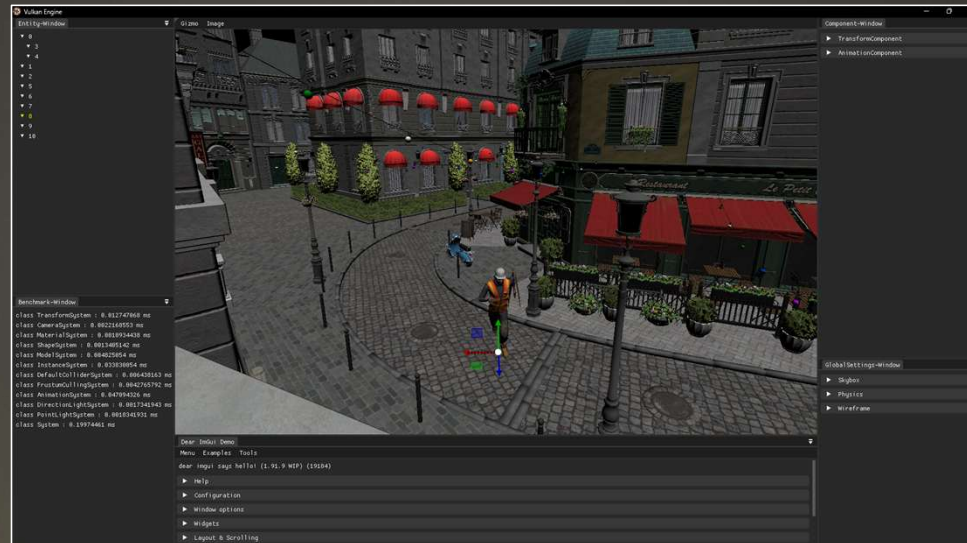


3D grafikus motor fejlesztése Vulkan alapokon

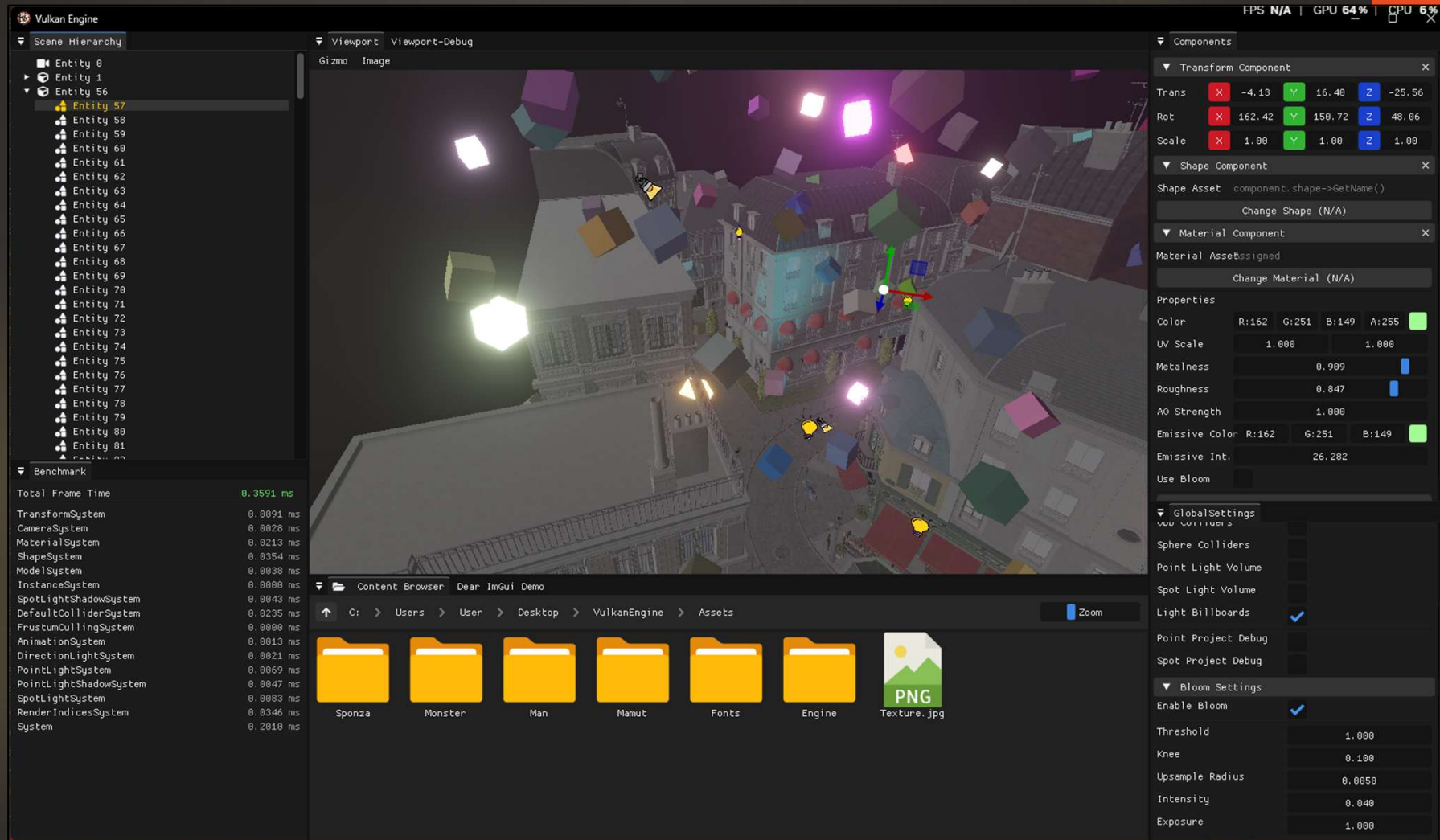
Készítette: Tamás Péter

Előző félév összefoglaló (Önlab I.)

- ▶ Engine + **Bindless Vulkan** architektúra
- ▶ Adatorientált **Entity-Component-System**
- ▶ Párhuzamos systemek és komponens feldolgozás (cpu/gpu)
- ▶ **Async managerek**: model, animáció, textúra betöltése
- ▶ Komplex **model/animáció rendering**
- ▶ **Vertex pulling** shaderek
- ▶ Cpu **frustum culling**, AABB, OBB, Sphere collider generálás
- ▶ **Deferred shading** G-Buffer, kezdeti fény szimuláció
- ▶ + Sok minden más :)



Játékmotor jelenlegi állapota (Önlab II.)



Billboarding

- ▶ Irány-, pont- és spotfényforráshoz mindig kamera felé néző iconok
- ▶ Geometry shaderben generálom a billboard felület vertexeit, a kamera paraméterek alapján.



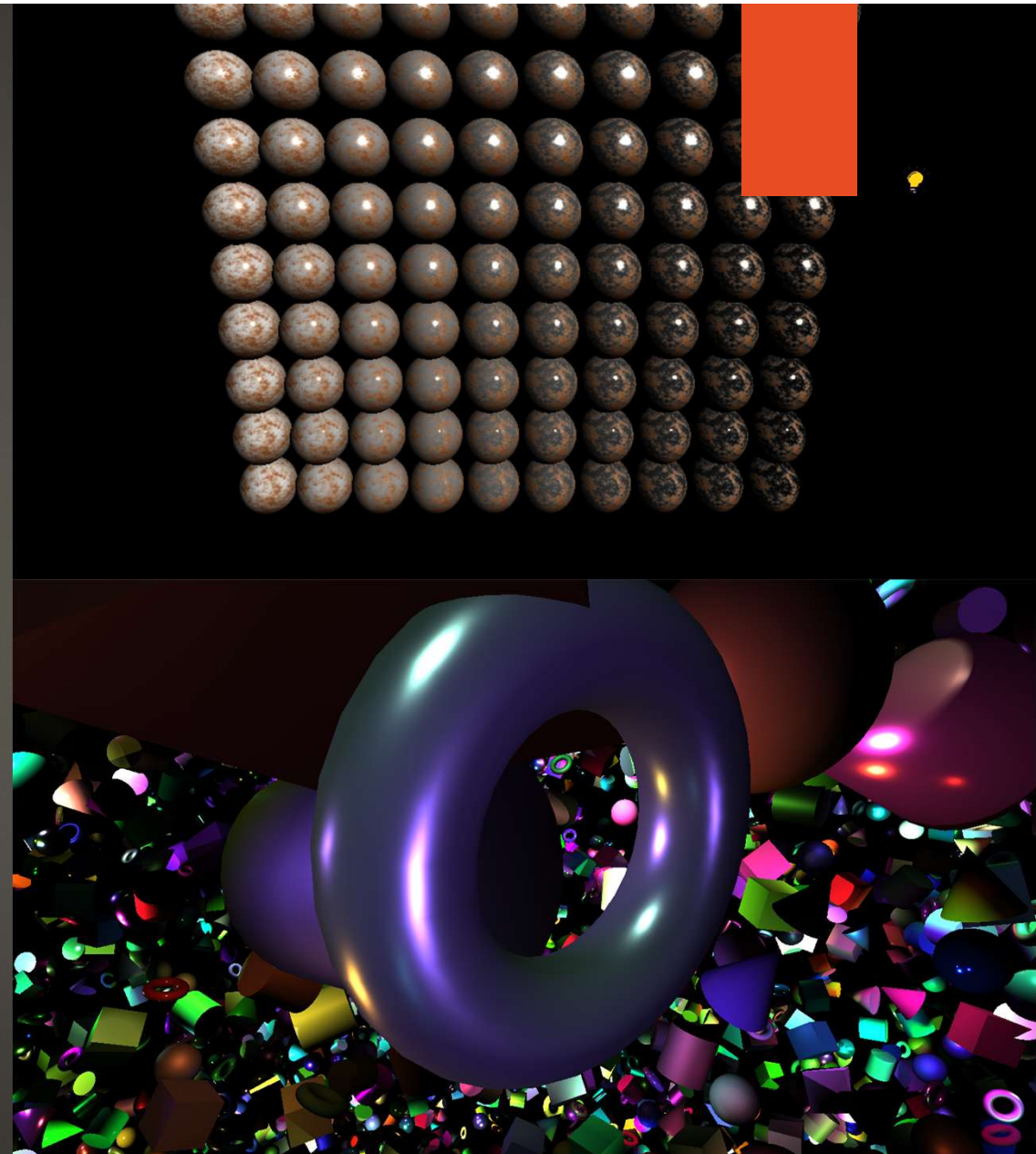
Deferred Shading

- ▶ **One-to-One blending** fekete textúrából indulva
- ▶ Irányfényforrásoknál fullscreen quad instanced draw
- ▶ Pontfényforrásoknál **CUBE CULLED INSTANCED INDIRECT** draw
- ▶ Spotfényforrásoknál **CONE CULLED INSTANCED INDIRECT** draw
- ▶ Tehát: Pont és Spot fény is **gpu-driven-culled** és **rendered!** (Erről majd később többet)
- ▶ + **Bloom/Ao pass!**



Physically Based Rendering (PBR)

- ▶ CookTorrance Brdf alapján
- ▶ Normal distribution function: **Trowbridge-Reitz GGX**
- ▶ Geometry function: **Schlick GGX**
- ▶ Fresnel equation: **Fresnel Schlick approximáció.**
- ▶ Material szinten: Albedo, Normal, Roughness, Metallic, Ao, Emissive textúrák és paraméterek
- ▶ Textúrák kombinálva (normal + roughness, hatékonyabb shader sample érdekében)



Gpu-Driven engine architektúra

► Culling compute shaderben GPU-n

1. Camera Frustum vs Object AABB culling
2. Camera Frustum vs Point light SPHERE culling
3. Camera Frustum vs Spot light AABB culling
4. Látható „entitások” Instance Index buffereinek előállítása instance renderinghez!
5. Látható cullolt Shadow Point light vs Object AABB culling (Shadow Map renderhez)
6. Látható cullolt Shadow Spot light vs Object AABB culling (Shadow Map renderhez)

EZEKET MIND COMPUTE SHADERBEN! :)

► Instanced Indirect „Multi” Draw renderelés (Minden render command GPU-n él!!)

1. Indirect Command Buffer előállítása GPU-n, culling compute shaderben
2. vkCmdDrawIndirectCount -> Gpu bufferben tárolt Indirect Commandok alapján indít render hívást.
(A **Count** egy vk extension, Gpu Bufferből olvassa vissza, hogy hány Indirect Commandot kezeljen: a 0 instance countos commandok szűrhetőek így!!)

Camera Frustum vs Object AABB (Compute Shader)

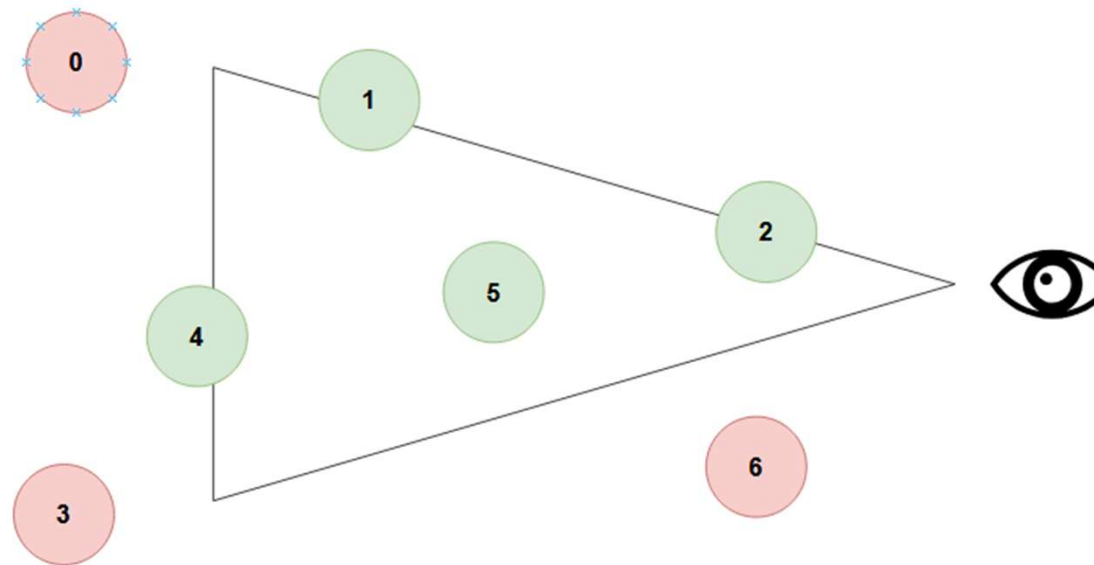
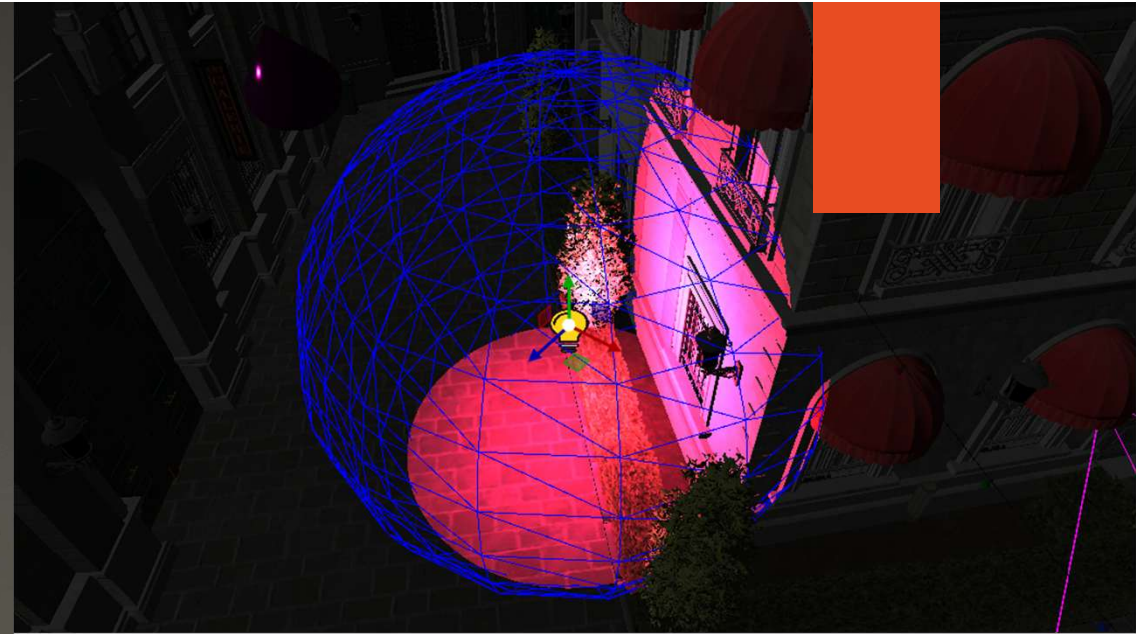
- ▶ **Compute Dispatch:** (Object Count, 1, 1)
- ▶ **atomicAdd:** Indirect Command Instance Count értékét növelem atomic módon. Visszadja az atomicAdd az előző count értéket -> Ez az index az Instance Index Bufferbe!
- ▶ **Instance Index Buffer:** Dinamikusan packed tárolja a látható entitások indexeit, ami rendereléshez kell
- ▶ Instance Index Buffer a fenti képhez [0, 2, 4, 6] -> Instance Count 4 db

(Feltesszük, hogy a compute shader invocations lineárisan végezett mindig!!, gyakorlatban ez nincs így persze)



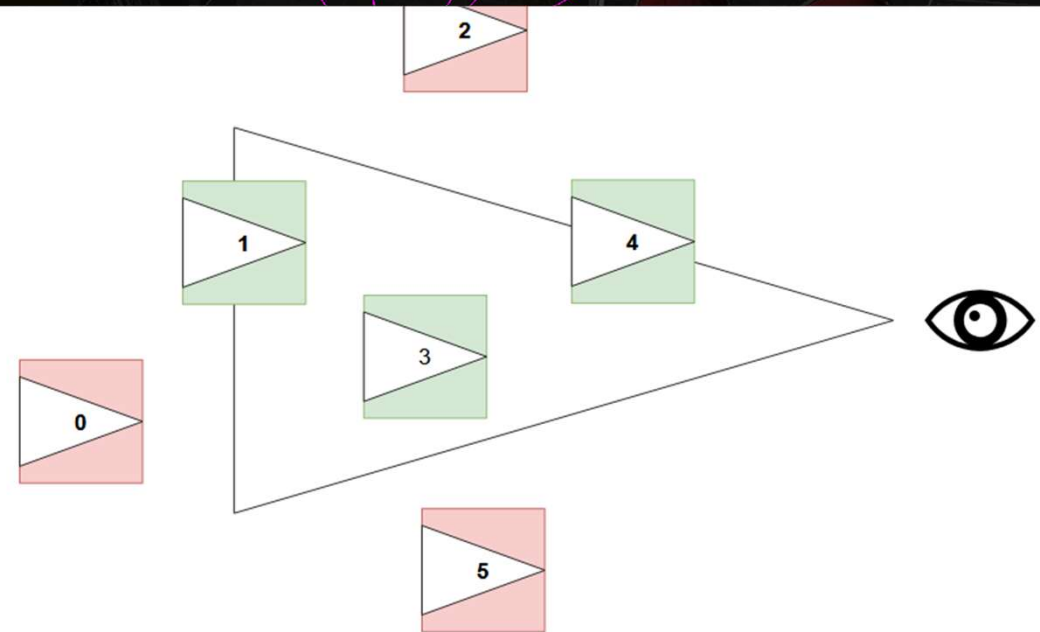
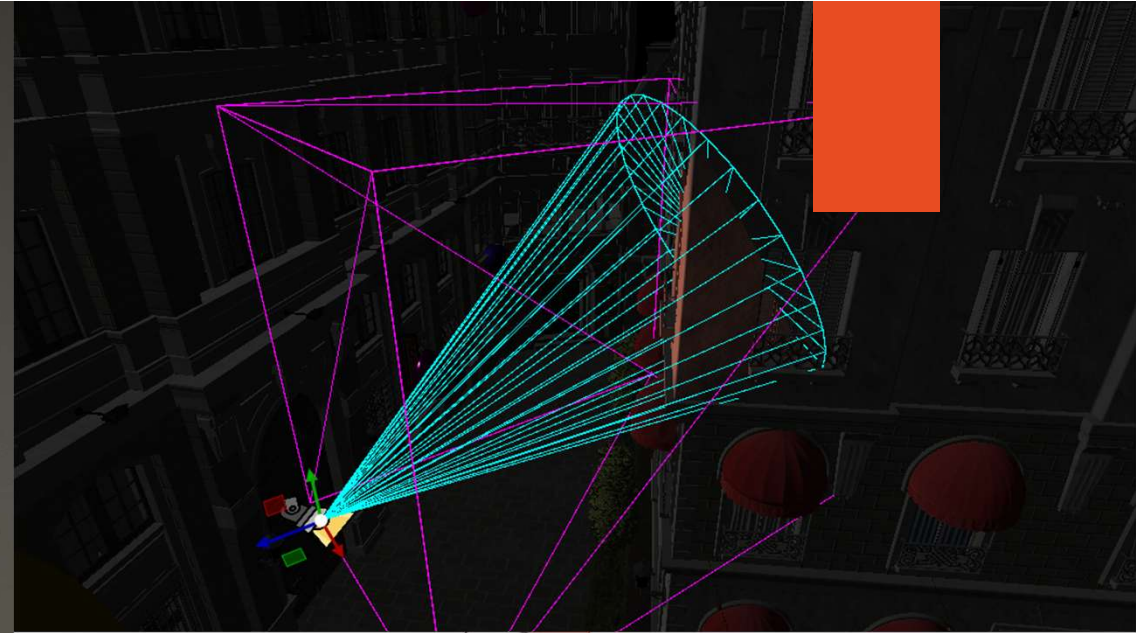
Camera Frustum vs Point Light Sphere (Compute Shader)

- ▶ **Compute Dispatch:** (Point Light Count, 1, 1)
- ▶ **atomicAdd:** ugyan az.
- ▶ **Instance Index Buffer:** ugyan az + **Deferred shading** pipelinehoz kell ez esetben. Csak azokat a pont fényforrásokat fogom renderelni és szimulálni ami látszik is.



Camera Frustum vs Spot light AABB (Compute Shader)

- ▶ **Compute Dispatch:** (Spot Light Count, 1, 1)
- ▶ **atomicAdd:** ugyan az.
- ▶ **Instance Index Buffer:** ugyan az + **Deferred shading** pipelinehoz kell ez esetben. Csak azokat a spot fényforrásokat fogom renderelni és szimulálni ami látszik is.
- ▶ Cone köré vett AABB alapján fog a metszés vizsgálat történni



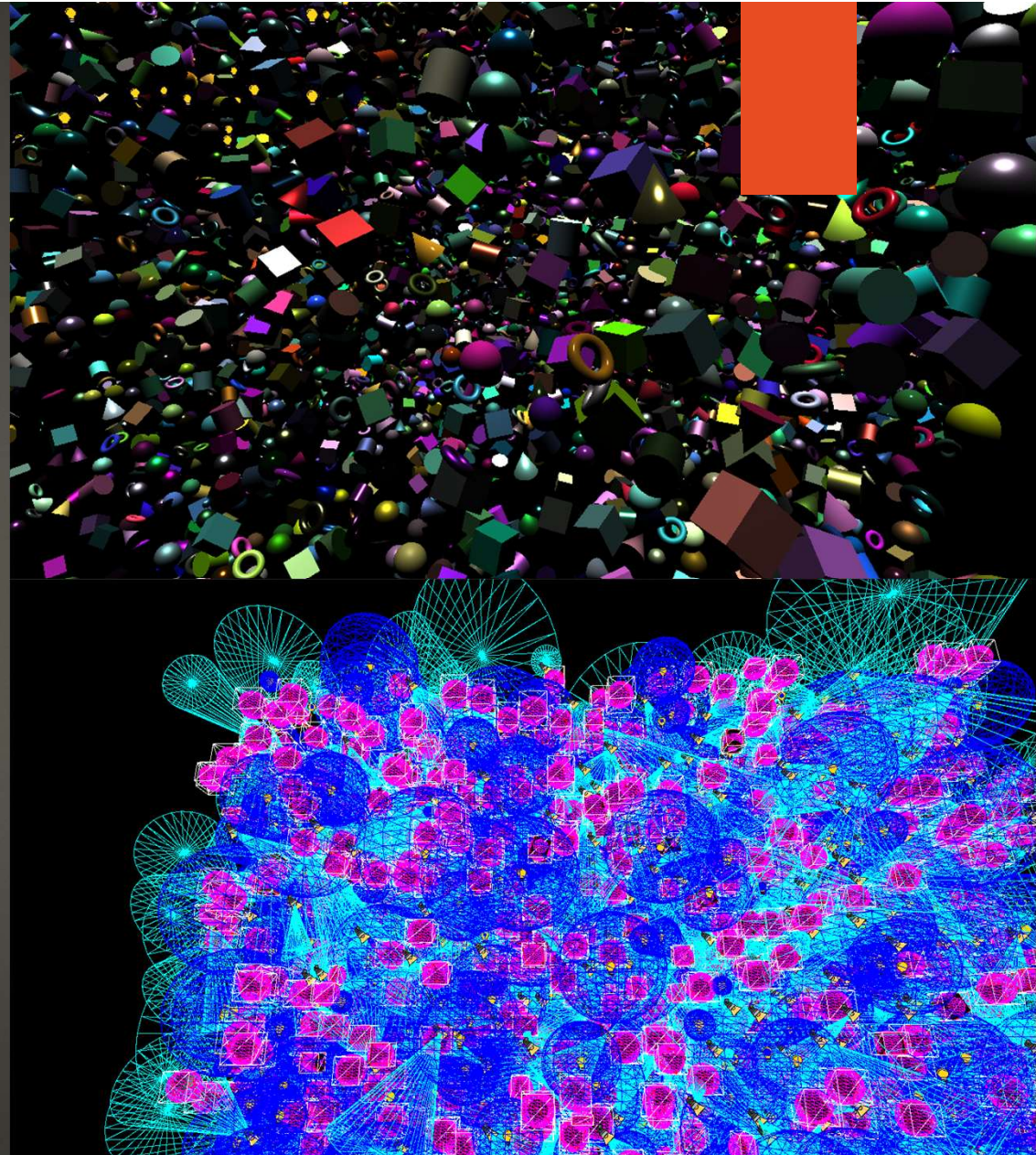
Gpu-Driven-Rendering

► Culling compute shaderben GPU-n előállt:

1. Object/Point/Spot light **Instance Index Bufferek** tartalma látható indexekkel.
2. Az **Indirect Draw Commandok**, az atomicAdd mechanizmussal az instanceCount a renderelendő számot tartalmazza (A többi paraméter előzetesen cpu oldalt már be van álltva)

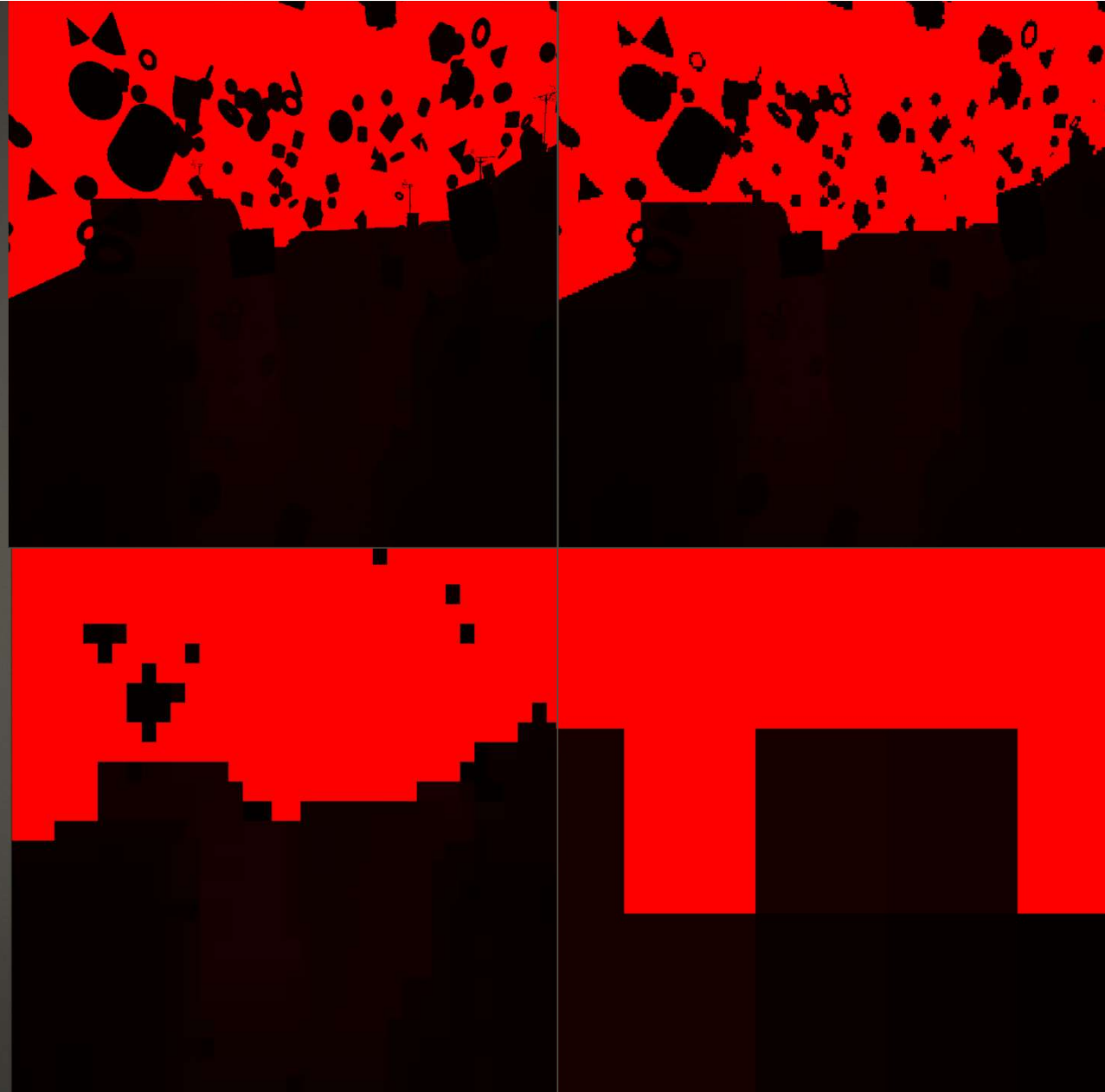
► Indirect ,Multi' Draw renderelés

1. **vkCmdDrawIndirectCount** -> 1 db render hívás, az összes különböző típusú objektum összes előfordulásának renderelése
2. **gl_DrawIDARB**: Indirect Draw azonosító a commandokhoz -> Innen lehet tudni melyik objektum a shaderben
3. **gl_InstanceIndex**: Az objektumhoz tartozó Instance Index Bufferbe lehet indexelni vele -> Megvan az „entitás” -> Transform stb... lekérhető a globális bufferekből
4. **Vertex Pulling** kötelező, anélkül nem lehetne megvalósítani!



Occlusion Culling HiZ

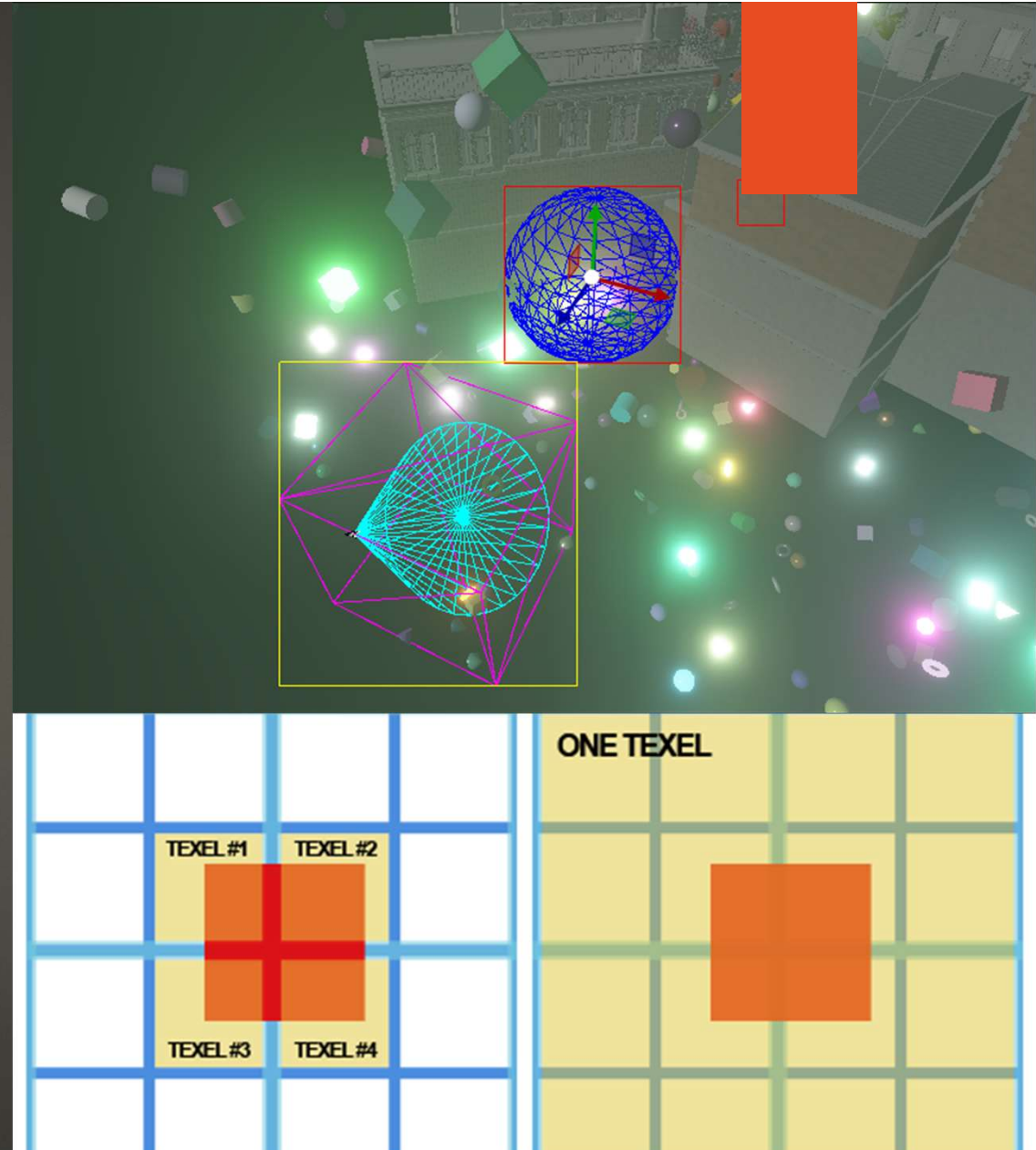
- ▶ Hierarchikus max reduction depth buffer mip chain textúrák.
- ▶ 2x2 régiók maximum mélységét vesszük folyamat fele akkora textúrákban. ->
Vulkan Max Reduction Sampler Extension
- ▶ **Object/Point/Spot culling compute shaderbe épül be!!.**
- ▶ Először frustum culling, majd a visible objektumokon occlusion culling (takarás vizsgálat)
- ▶ **Push Descriptor** használata a src/dst compute shader input hiz mip textúrákhoz



Occlusion Culling folytatás...

- ▶ AABB esetén: **AABB projektálni Screen Space-re**, min/max meghatározása -> 2D AABB
- ▶ Sphere esetén: **Sphere projektálni Screen Space** (Van perspektív projekciós torzítás!!), min/max meghatározása -> 2D AABB
- ▶ **A projektált 2D AABB:**
 1. **Középpontja**: Textúra mintavételezési pont.
 2. **X/Y mérete**: HiZ mip sample level határozható meg vele!

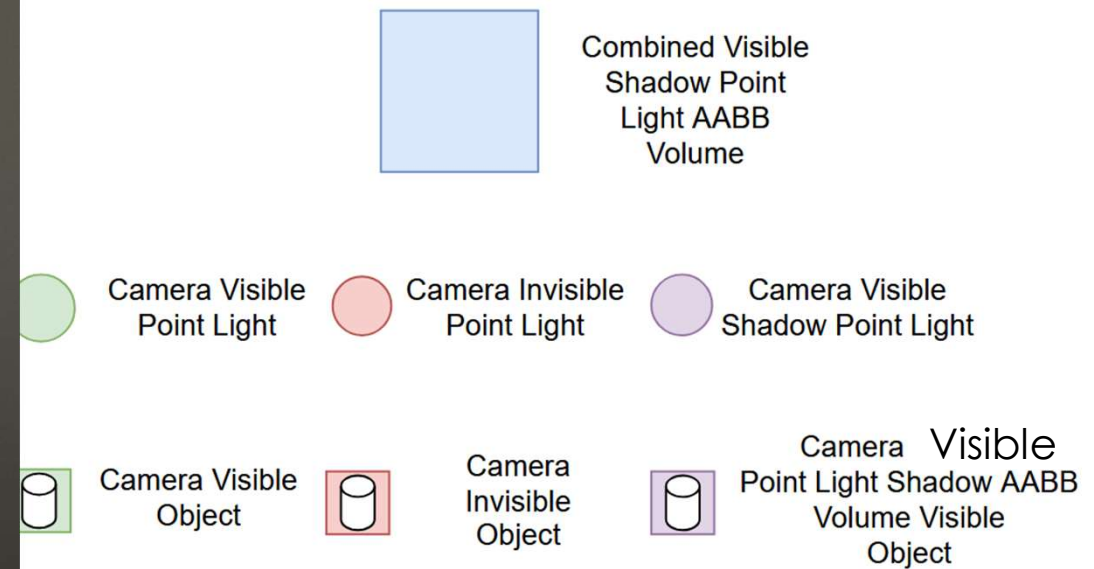
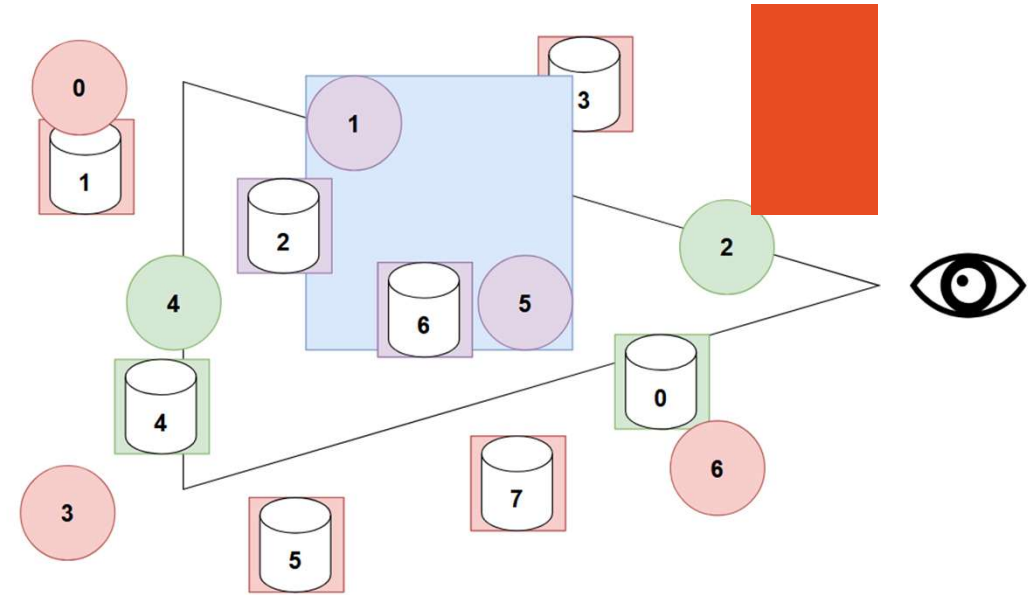
Fontos: **Méret** – 1 levelről kell max **reduction samplerrel olvasni!** -> Különböző side effectek!



Gpu-Driven-Culling folytatás...

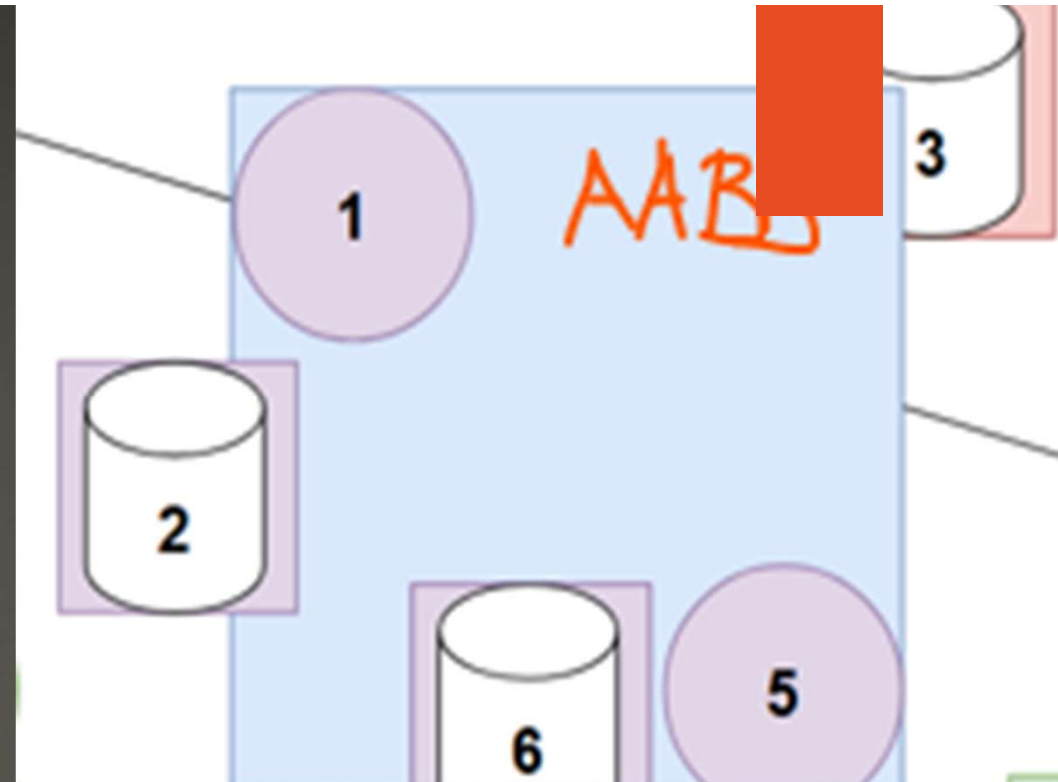
Point/Spot shadow light AABB volume vs Objects

- ▶ Point és Spot shadow map generáláshoz is lehetne **compute shader culling**, **indirect draw renderinget** használni.
- ▶ Az ötlet ugyan az, mint a camera frustum esetén, csak most shadow point lightokra / shadow spot lightokra cullolunk objektumokat, és építünk Instance Index Buffert, illetve Indirect Draw Commandokat
- ▶ A Point/Spot culling compute shaderben határozom meg a shadow visible lightokat -> Erre épülnek további compute shaderek

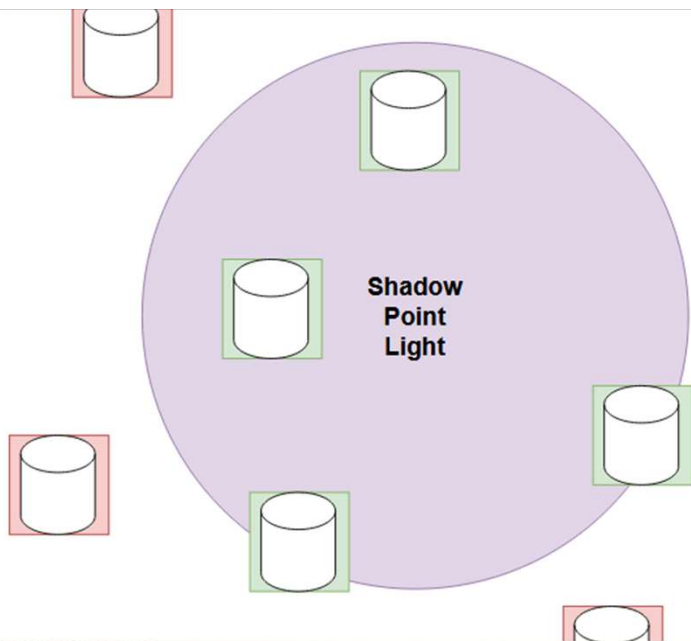


Point/Spot shadow AABB optimalizálás

- ▶ **Ne!!:** Shadow vs All Object compute shader dispatch dimenziók legyenek! Túl sok felesleges teszt lenne.
- ▶ **Optimalizálás:**
 1. Előzetesen a látható shadow lightok **közös volume-ának AABB** meghatározása.
 2. Az összes objektum cullolása az AABB-re, és indexek elmentése bufferbe.
(NxM helyett így NxK, ahol K a volume-mal metszésben álló objectum szám + A volume nem sokkal nagyobb a frustumnál a legrosszabb esetben, vagy sokkal kisebb is lehet amúgy!!)
 3. Ezek után egyedileg a shadow lightokra az előző pontban cullolt objektum cullolása.
(**Instance Index Buffer + Indirect Draw Command** itt jön létre)
- ▶ **Float min/max shaderben:** UINT növekvő ábrázolásra kell alakítani a float értékeket, majd UINT-re van beépített max/min függvény. Végül az AABB paraméterek visszaalakítani float-ra!



```
uint floatToSortableUint(float f) {  
    uint u = floatBitsToUint(f);  
    uint mask = uint(-int(u >> 31)) | 0x80000000u;  
    return u ^ mask;  
}  
  
float sortableUintToFloat(uint u) {  
    uint mask = ((u >> 31) - 1u) | 0x80000000u;  
    return uintBitsToFloat(u ^ mask);  
}
```



Point/Spot shadows vs Objektum compute culling

- ▶ Pontfényforrás esetén: **Sphere vs AABB**
- ▶ Spotfényforrás esetén: **AABB vs AABB**
- ▶ **Compute Command Dispatch** (N, M, 1):
 1. **N** = Látható shadow light száma
 2. **M** = Shadow AABB volume-mal metszésben álló objektumok száma

FONTOS: N és M értékeket is előzetesen a culling compute shaderekben jöttek létre -> Ezért **Indirect Dispatch Commandra** van szükség.

Vagyis: A compute shader dispatch dimenziókat egy GPU bufferből fogja megkapni a vulkan dispatch hívás
-> Ugyan az a lényege, mint az Indirect Drawnak!

Phase 1 Dispatch: (1, 6, 1)

▼ Shadow #0 (M: 2, S: 1)

MODELS

- ▶ C:/Users/User/Desktop/Bistro_v5_2/BistroInterior.fbx (1)
- ▶ C:/Users/User/Desktop/Bistro_v5_2/BistroExterior.fbx (1)

SHAPES

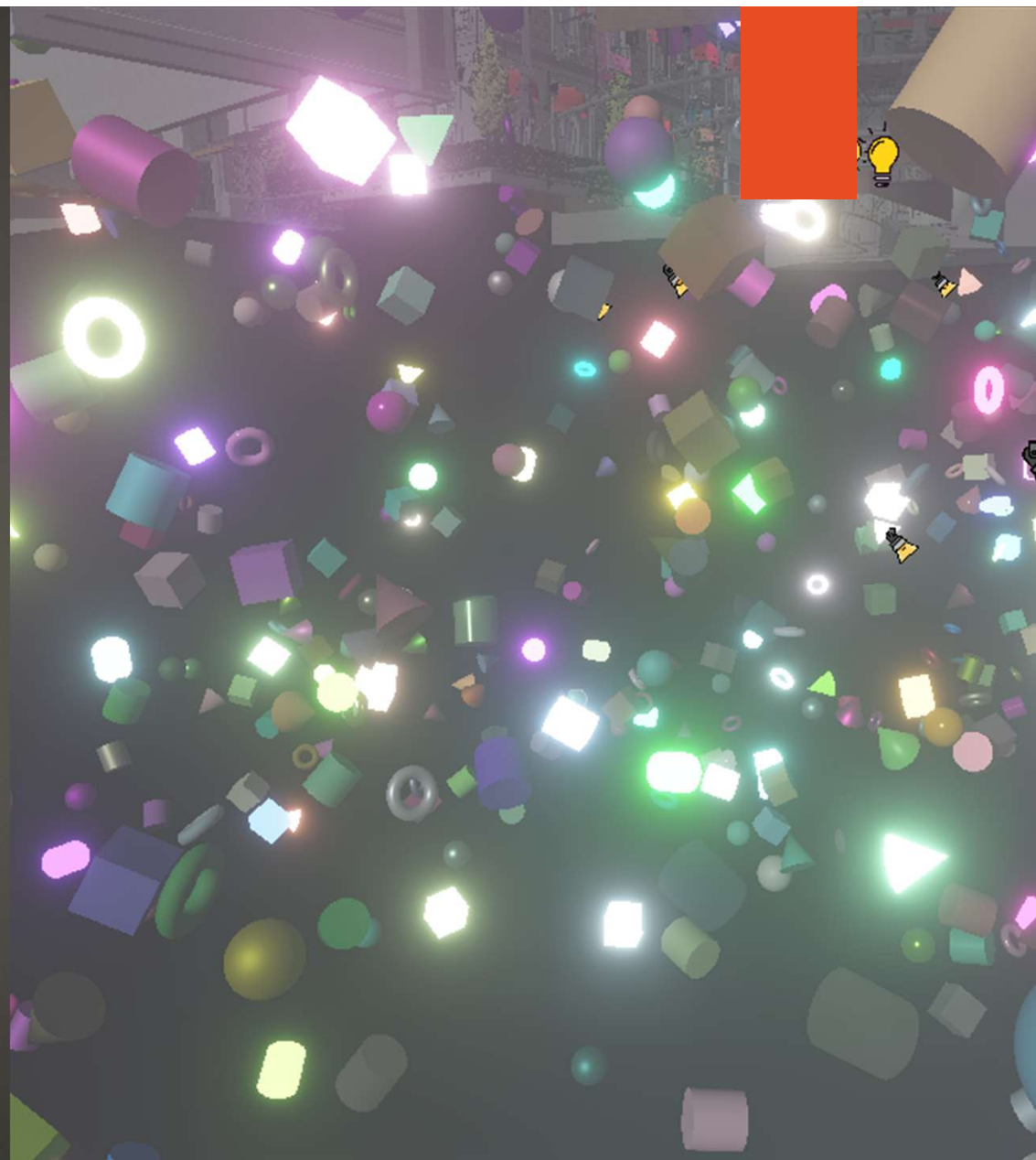
▼ Cube (129)

Indices: [32, 33, 36, 40, 42, 30, 49, 60, 63, 57, 58, 45, 48, 220, 221, 55, 56, 223, 96, 44, 217, 219, 244, 245, 103, 104, 246, 106, 241, 242, 243, 1, 2, 3, 125, 127, 179, 122, 123, 75, 248, 65, 68, 102, 66, 70, 71, 195, 109, 111, 72, 73, 184, 185, 112, 232, 191, 118, 120, 238, 239, 193, 129, 130, 203, 204, 82, 97, 98, 19, 133, 99, 100, 188, 189, 77, 24, 26, 139, 142, 27, 28, 144, 8, 9, 225, 227, 207, 140, 149, 213, 214, 147, 11, 150, 153, 205, 206, 220, 229, 12, 174, 199, 200, 78, 79, 15, 16, 154, 196, 236, 209, 85, 86, 4, 6, 169, 170, 201, 87, 163, 178, 157, 158, 224, 83, 84, 164, 165]

▶ Shadow #1 (M: 0, S: 0)

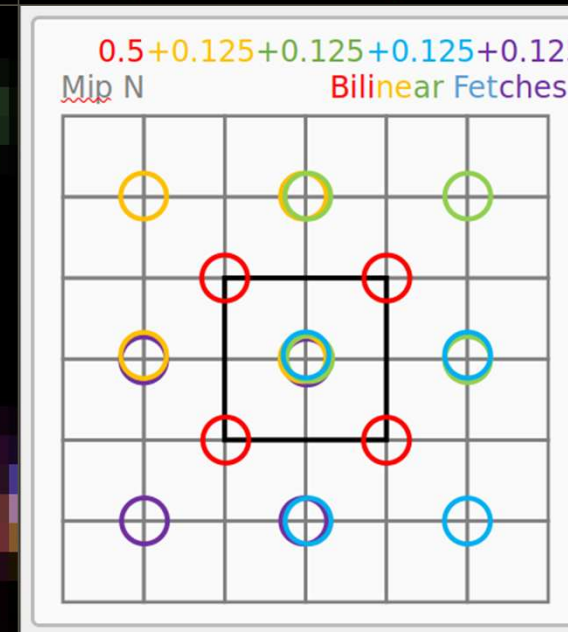
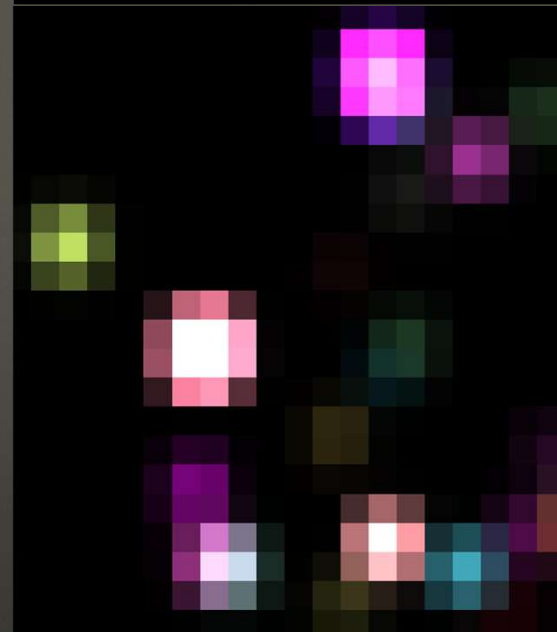
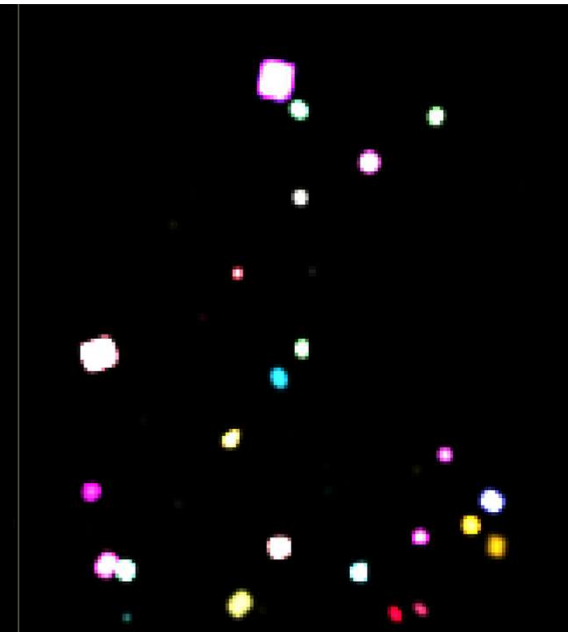
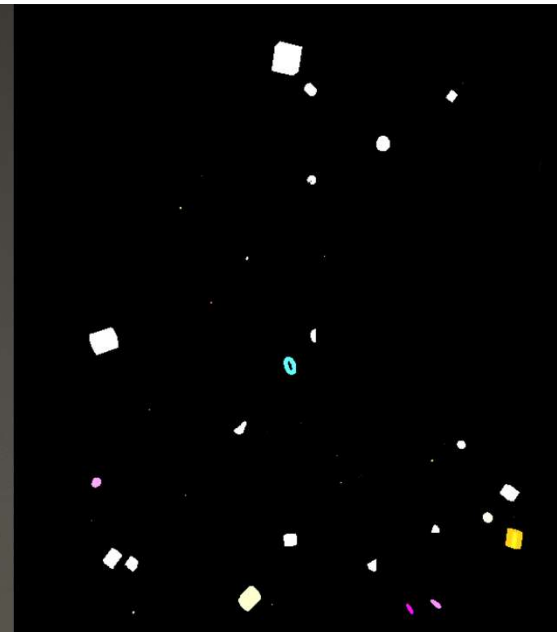
Bloom simulation

- ▶ Unity/CoD algoritmus:
Bloom textúra mip chain készítése,
és súlyozott környezet átlagolása.
- ▶ Külön Bloom textúrába menteni a
magas intenzitású színértékeket
- ▶ **Down-Sample:** Kisebb mipekre
szűkítjük a textúrát és tartalmát
- ▶ **Up-Sample:** Egymásra mossuk a
felskálázott textúrát
- ▶ **Végül:** Main textúra + Bloom mix
tone mappelés

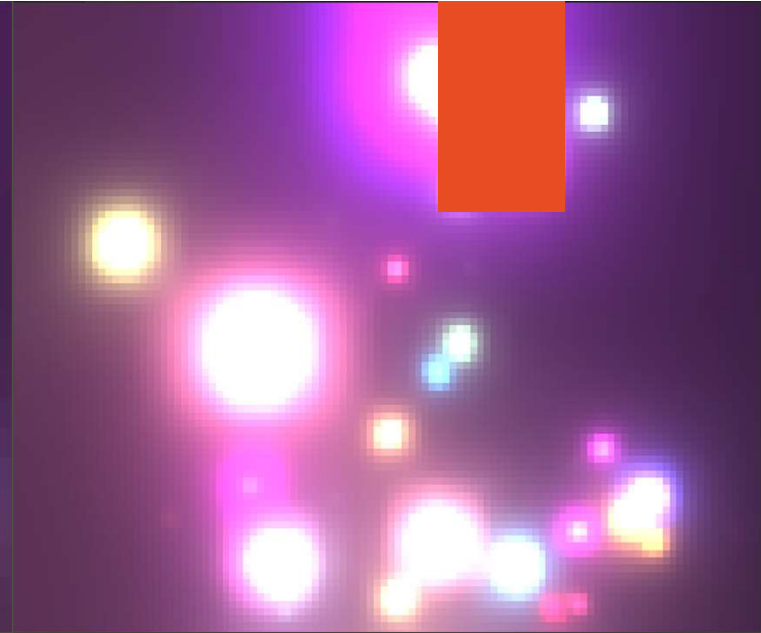
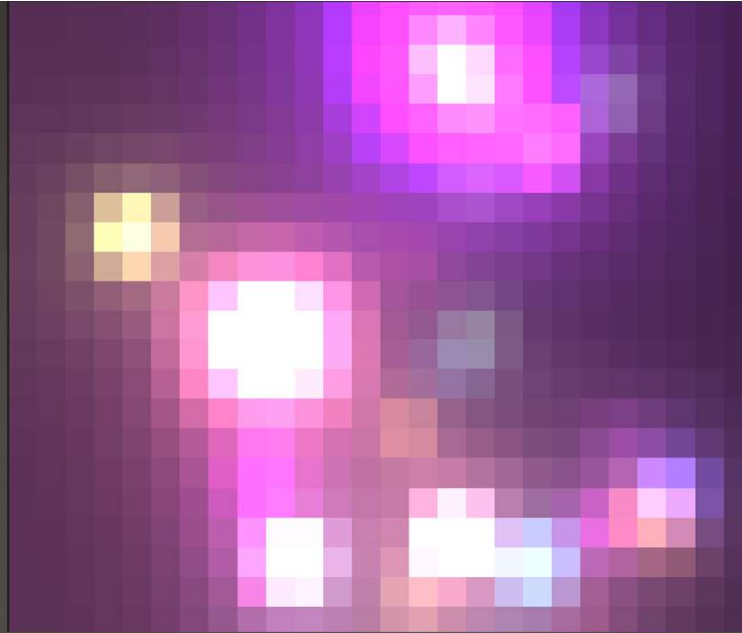


Bloom Down-Sample pass (Compute Shaderben)

- ▶ Adott pixel körül fix helyeken mintavételezünk többször is, és a mintavételezési pontokat súlyozzuk, majd összegezzük!
- ▶ **Push Descriptor** használok a src/dst imagek feltöltésére compute shaderbe



Bloom
Up-Sample pass
(Compute
Shaderben)



ImGui UI ablakok fejlesztése...

